



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Tyrael Lance Peralta
Section:	AN42
Professor:	Crisola Tan
Date:	10/03/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
(Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
 - Classes and objects
 - If-Else statements

- Loops (for loop)
- Functions / methods
- 2. Understand the environment:
 - Two rooms: A and B
 - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output

```
C:\Users\hazel\AppData\Local... x + v
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Press any key to continue . . . |
```

Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
 - Rule 1: If the current room is dirty, clean it.
 - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
 - The agent's current location
 - The action taken (cleaned/moved)
 - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)

```
C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe C:\Users\hazel\AppData\Local\Programs\Python\Python39-64\python.exe
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room B
[A-Clean] [B-Agent] [C-Dirty]

Step 2:
[A-Clean] [B-Agent] [C-Dirty]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Dirty]

Step 3:
[A-Clean] [B-Agent] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]

Step 4:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]

Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press any key to continue . . .
```

Your Code - Laboratory Tasks

```
import agentpy as ap
import random
```

```
# Ask the user for a valid room condition
```

```
def ask_room_condition(room_name):
    while True:
        condition = input(
            f'Is Room {room_name} Dirty or Clean? '
        ).strip().lower()
```

```

    if condition in ["dirty", "clean"]:
        return condition

    print("Please enter only 'Dirty' or 'Clean'.")

# Get user input
room_a = ask_room_condition("A")
room_b = ask_room_condition("B")
room_c = ask_room_condition("C")
num_steps = int(input("\nHow many steps should the agent run? "))

# Agent Definition
class RoomCleaner(ap.Agent):

    def setup(self):
        self.current_room = "A"
        self.actions = 0

    def step(self):
        self.actions += 1
        room_status = self.model.rooms[self.current_room]

        if room_status == "dirty":
            print(
                f"\n\nStep {self.actions}: "
                f"Room {self.current_room} is dirty. Cleaning it."
            )

            self.model.rooms[self.current_room] = "clean"

        else:
            old_room = self.current_room

            # Select another room randomly
            possible_rooms = [
                room for room in self.model.rooms
                if room != self.current_room
            ]

            self.current_room = random.choice(possible_rooms)

            print(
                f"\n\nStep {self.actions}: "

```

```

        f"Room {old_room} is clean. "
        f"Moving to Room {self.current_room}."
    )

```

Model Definition

class Rooms(ap.Model):

def setup(self):

```

    self.rooms = {
        "A": room_a,
        "B": room_b,
        "C": room_c
    }

```

Create one RoomCleaner agent

self.cleaner = RoomCleaner(self)

```

print("\nInitial Room Conditions:")
print(f"Room A: {self.rooms['A']}")
print(f"Room B: {self.rooms['B']}")
print(f"Room C: {self.rooms['C']}")
print(f"Agent starts in Room A.\n")

```

def step(self):

```

    self.cleaner.step()
    print(f"Agent current position:")
    print( "[ A | B | C ]")
    print(f"[ {self.rooms['A']} | {self.rooms['B']} | {self.rooms['C']} ]")
    if self.cleaner.current_room == 'A':
        print( "[ 0 | - | - ]")

```

```

    elif self.cleaner.current_room == 'B':
        print( "[ - | 0 | - ]")

```

```

    elif self.cleaner.current_room == 'C':
        print( "[ - | - | 0 ]")

```

def end(self):

```

    print("\n\nSimulation Finished.")
    print("Final Room Conditions.")
    print(f"[ A - {self.rooms['A']} | B - {self.rooms['B']} | C - {self.rooms['C']} ]")
    print(f"Agent's final location: Room {self.cleaner.current_room}")

```

```

# Simulation parameters
parameters = {
    "steps": num_steps
}

# Create and run the model
model = Rooms(parameters)
model.run()

```

OUTPUT:

```

Is Room A Dirty or Clean? dirty
Is Room B Dirty or Clean? dirty
Is Room C Dirty or Clean? dirty

How many steps should the agent run? 10

Initial Room Conditions:
Room A: dirty
Room B: dirty
Room C: dirty
Agent starts in Room A.

Step 1: Room A is dirty. Cleaning it.
Agent current position:
[ A | B | C ]
[ clean | dirty | dirty ]
[ 0 | - | - ]
Completed: 1 steps

Step 2: Room A is clean. Moving to Room B.
Agent current position:
[ A | B | C ]
[ clean | dirty | dirty ]
[ - | 0 | - ]
Completed: 2 steps

Step 3: Room B is dirty. Cleaning it.
Agent current position:
[ A | B | C ]
[ clean | clean | dirty ]
[ - | 0 | - ]
Completed: 3 steps

Step 4: Room B is clean. Moving to Room A.
Agent current position:
[ A | B | C ]
[ clean | clean | dirty ]
[ 0 | - | - ]
Completed: 4 steps

Step 5: Room A is clean. Moving to Room C.
Agent current position:
[ A | B | C ]
[ clean | clean | dirty ]
[ - | - | 0 ]
Completed: 5 steps

Step 6: Room C is dirty. Cleaning it.
Agent current position:
[ A | B | C ]
[ clean | clean | clean ]
[ - | - | 0 ]
Completed: 6 steps

Step 7: Room C is clean. Moving to Room A.
Agent current position:
[ A | B | C ]
[ clean | clean | clean ]
[ 0 | - | - ]
Completed: 7 steps

Step 8: Room A is clean. Moving to Room C.
Agent current position:
[ A | B | C ]
[ clean | clean | clean ]
[ - | - | 0 ]
Completed: 8 steps

Step 9: Room C is clean. Moving to Room A.
Agent current position:
[ A | B | C ]
[ clean | clean | clean ]
[ 0 | - | - ]
Completed: 9 steps

Step 10: Room A is clean. Moving to Room B.
Agent current position:
[ A | B | C ]
[ clean | clean | clean ]

Simulation Finished.
Final Room Conditions.
[ A - clean | B - clean | C - clean ]
Agent's final location: Room B

Run time: 0:00:00.006687
Simulation finished

```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ↓

1. If the room is dirty then clean it
If not then move to another random room
2. The agent will clean if the room is dirty but if its already clean then it will move
3. The agent will move between them until all steps are done
4. I would modify it so the agent will go each rooms in order rather than random, I could also put room with higher priorities to be cleaned first before the others.